SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Project: Online Bookstore

Version: 1.0

Team	USN	Name
Team 4	PES2UG24CS076	Anvii S Rai
Team 4	PES2UG24CS077	Anwitha Potluri
Team 4	PES2UG24CS078	Apoorva Kogta
Team 4	PES2UG24CS079	Arayana Jain

Technology Stack: JavaScript (MERN Stack) / Python (Django)

Date: 04-10-2026
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 4	Initial architecture and design specification for the Online Bookstore

Approvals

Role	Name	Signature / Date
Course Coordinator		
Project Mentor		

1. Introduction

1.1 Purpose

This document describes the software architecture and design of the Online Bookstore. It explains how the main parts of the application are organised, how they communicate, and how the design supports the requirements given in the SRS.

1.2 Scope

The design covers the customer and guest functions shown in the UML use-case model, including authentication, browsing and searching books, cart and wishlist management, checkout, payment, order tracking and reviews. It also covers administrator functions such as book, category, inventory, order, user and review management.

1.3 Audience

The document is intended for the project team, developers, testers, project mentor and anyone reviewing or maintaining the application.

1.4 Definitions

- API – Application Programming Interface used for communication between application components.
- JWT – JSON Web Token used for authenticated requests.
- RBAC – Role-Based Access Control used to restrict administrator operations.
- MERN – MongoDB, Express.js, React and Node.js.
- REST – Representational State Transfer style used for the web APIs.
- ADR – Architecture Decision Record.

2. Document Overview

2.1 How to Use This Document

The document first gives the overall architecture and the reasons behind the main design choices. It then describes the components, data and security design, followed by detailed design flows, API examples, error handling and deployment. The diagrams are intended to give a quick view of how the system is put together.

2.2 Related Documents

- Online Bookstore Software Requirements Specification (SRS) v1.0.
- Online Bookstore Software Test Plan (STP) v1.0.
- Requirements Traceability Matrix (RTM).

- Customer and Guest Use-Case Diagram.
- Administrator Use-Case Diagram.

3. Architecture

3.1 Goals & Constraints

The architecture is kept simple enough for a student project while still separating the frontend, backend, authentication, business logic and database responsibilities.

Goal / Constraint	Design Response
Maintainability	Separate UI, API, business logic and data access responsibilities.
Security	Use authenticated APIs, role checks, password hashing and HTTPS in deployment.
Reliability	Validate operations before changing order or stock data and handle external service failures.
Usability	Keep customer flows straightforward and return readable validation/error messages.
Technology constraint	Use the selected JavaScript/MERN stack and/or Django backend as stated in the SRS.
External dependency	Payment and notification services are accessed through APIs or sandbox/mock services during development.

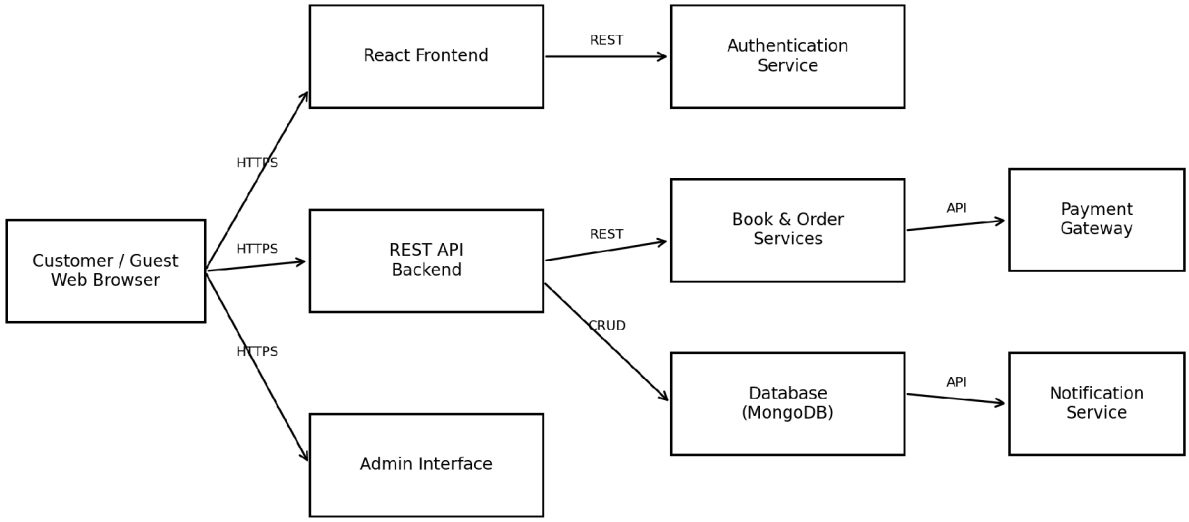
3.2 Stakeholders & Concerns

Stakeholder	Main Concern
Customer / Guest	Simple browsing and purchasing flow, correct order information and account security.
Administrator	Easy catalogue, inventory and order management with restricted access.
Development Team	Clear module boundaries and an architecture that is practical to implement and test.
QA / Tester	Traceable requirements, predictable APIs and testable flows.
Project Mentor / Evaluator	Clear separation of concerns and consistency with the SRS and UML models.

3.3 Component (UML) Diagram

The component view separates the browser interfaces from the application API, authentication, book/order services and database. External payment and notification services are kept outside the application boundary.

Online Bookstore – Component Diagram



3.4 Component Descriptions

Component	Responsibility
Customer / Guest Web Browser	Provides the customer-facing pages for browsing, account actions, cart, checkout and orders.
React Frontend	Handles page rendering, forms, client-side validation and calls to backend APIs.
Admin Interface	Provides protected screens for catalogue, inventory, users, orders, reviews and reports.
REST API Backend	Receives requests, validates input and coordinates authentication and business operations.
Authentication Service	Handles registration/login verification, password checks and session/JWT-related operations.
Book & Order Services	Contains catalogue, cart, checkout, order and review business rules.
Database	Stores users, books, categories, carts, orders, reviews and related application data.
Payment Gateway	External service used to authorise payments in the checkout flow.
Notification Service	Optional external service used for order/account notifications.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is used for the project. The main layers are presentation, API/application logic, business services and data access. This is easier for the team to implement and test than a full microservices setup.

A microservices architecture was not selected because the project is relatively small and the additional deployment, service discovery, monitoring and communication overhead would not provide enough benefit at this stage. The components are kept modular so that individual services can still be separated later if the application grows.

3.6 Technology Stack & Data Stores

Layer	Technology / Choice	Purpose
Frontend	JavaScript / React	Customer and administrator web interfaces.
Backend	Node.js + Express and/or Python Django	REST APIs and application logic.
Database	MongoDB	Users, books, categories, carts, orders and reviews.
Authentication	JWT or equivalent secure session mechanism	Authenticated access and role checks.
Communication	REST + JSON over HTTPS	Frontend-backend and external API communication.

External services	Payment gateway / notification API	Payment authorisation and optional notifications.

3.7 Risks & Mitigations

Risk	Mitigation
Payment service unavailable	Use sandbox/mock responses during development and handle failure states explicitly.
Incorrect stock during concurrent orders	Validate stock before order creation and update inventory as part of the order workflow.
Unauthorised administrator access	Use authentication plus server-side role checks for every protected admin operation.
Database/API failure	Return controlled errors and avoid marking an order successful until required operations are confirmed.
Scope becoming too large	Keep the first release aligned with the SRS and defer non-essential enhancements.

3.8 Traceability to Requirements

Requirement Group	Architectural Component / Layer
OBS-F-001 to OBS-F-003 – Authentication	React Frontend → REST API → Authentication Service → Database
OBS-F-004 to OBS-F-007 – Catalogue/Search	React Frontend → REST API → Book Service → Database
OBS-F-008 to OBS-F-010 – Cart/Wishlist	React Frontend → REST API → Cart/Book Service → Database
OBS-F-011 to OBS-F-014 – Checkout/Orders	React Frontend → REST API → Order Service → Database + Payment Gateway
OBS-F-015 – Reviews	React Frontend → REST API → Review logic → Database
OBS-F-016 to OBS-F-018 – Administration	Admin Interface → REST API → RBAC + Admin/Book/Order Services → Database
OBS-SR-001 to OBS-SR-005 – Security	Authentication, RBAC, API validation, HTTPS and controlled logging
OBS-NF-001 to OBS-NF-005 – Quality	API design, database design, error handling, responsive UI and deployment configuration

3.9 Security Architecture

Security is treated as part of the normal request flow rather than as a separate feature. Authentication is performed before protected operations, and administrator endpoints additionally check the user's role.

Threat / Concern	Example Risk	Design Response
Spoofing	Someone attempts to use another user's account.	Password hashing, authentication and token/session validation.
Tampering	A client changes an order or price in a request.	Server-side validation and business rules; never trust client totals or roles.
Information Disclosure	Sensitive account or payment information is exposed.	HTTPS, restricted responses and no sensitive values in logs.
Denial of Service	Repeated invalid or expensive requests affect availability.	Input validation, request limits where appropriate and monitoring.
Elevation of Privilege	A normal customer accesses admin functions.	Server-side RBAC checks on every admin endpoint.

4. Design

4.1 Design Overview

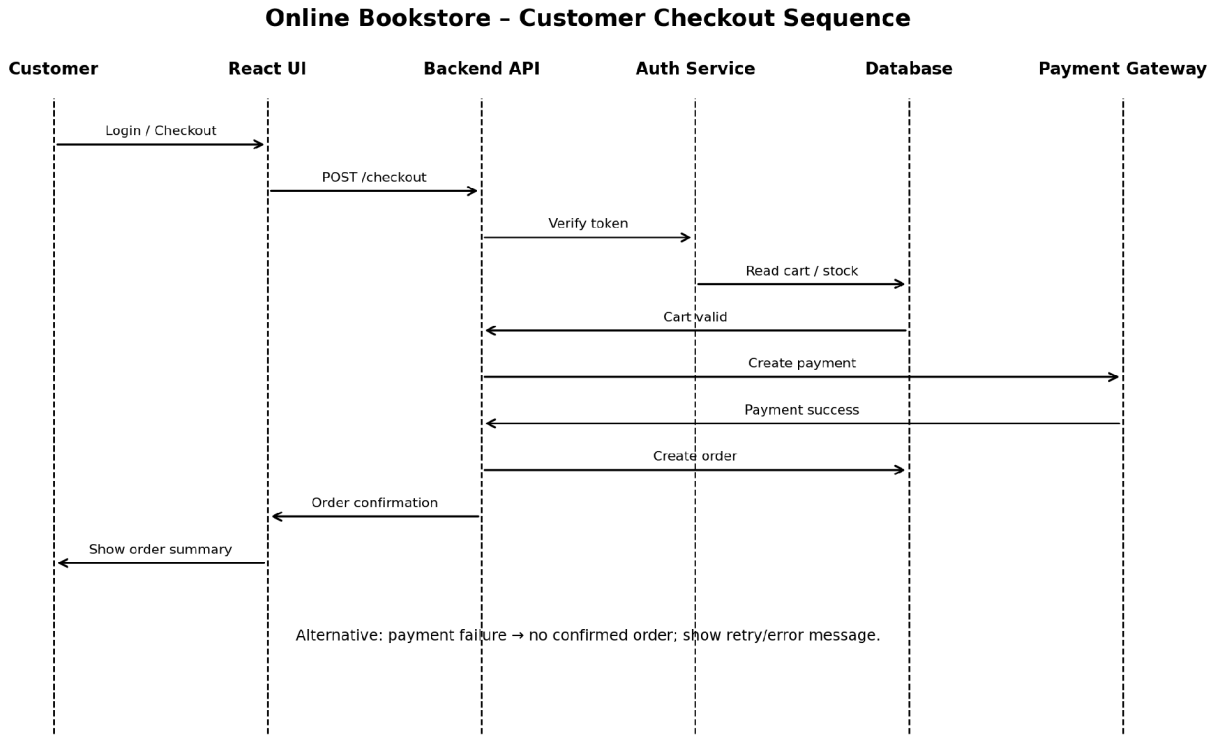
The design follows the same separation shown in the component view. The browser is responsible for presentation, while the backend remains responsible for authentication, validation and business rules. The database is accessed through backend services rather than directly from the browser.

For an order, the backend checks the authenticated user, validates the cart and stock, communicates with the payment service, and records the confirmed order. This keeps important business decisions on the server side.

4.2 UML Sequence Diagrams

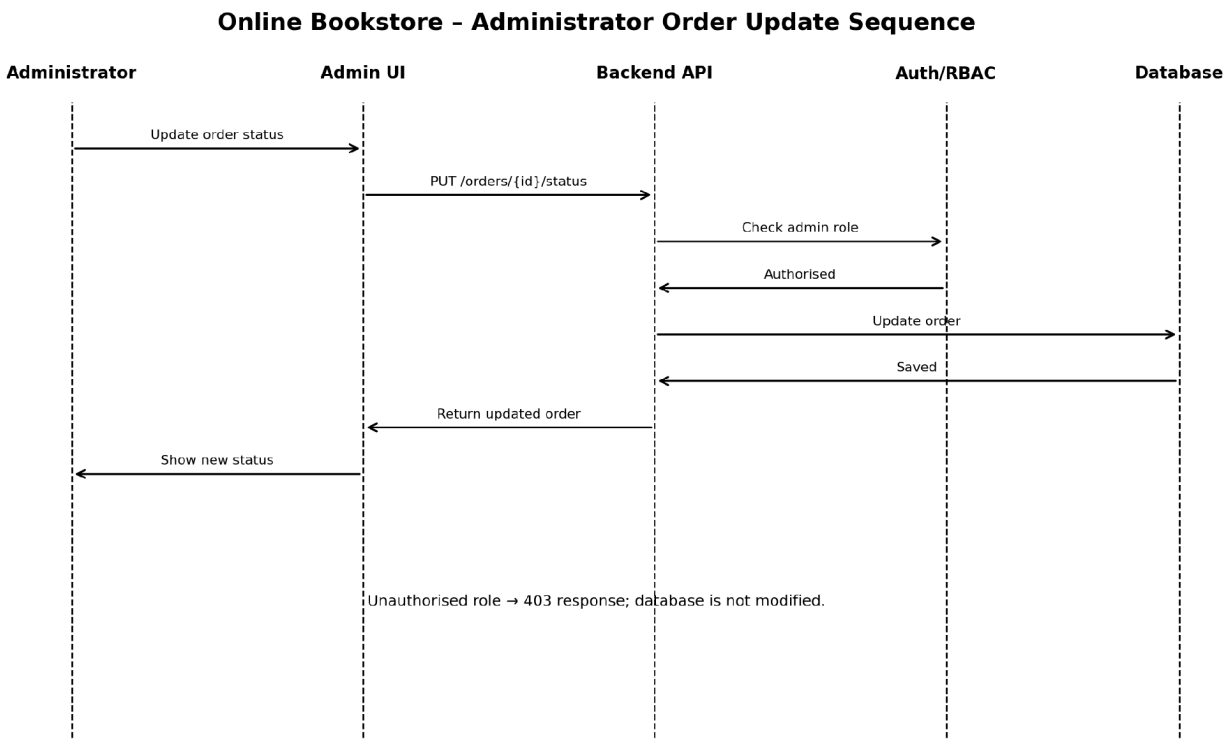
The sequence diagrams use standard UML-style lifelines and message arrows. They show two important project flows: customer checkout and administrator order-status update.

4.2.1 Customer Checkout



Main flow: the customer starts checkout, the frontend sends the request to the backend, authentication is verified, cart and stock are checked, payment is requested, and the order is stored only after a successful payment response. A failed payment returns an error/retry state.

4.2.2 Administrator Order Update



Main flow: the administrator submits a status change, the backend verifies the administrator role, updates the order and returns the updated status. An unauthorised request is rejected before the database is changed.

4.3 API Design

The API is organised around resources such as users, books, cart, orders and reviews. Requests and responses use JSON. Authentication information is sent using the configured session/JWT mechanism.

Endpoint	Method	Purpose	Typical Response / Errors
/api/auth/register	POST	Create a new customer account.	201 Created; 400 validation error.
/api/auth/login	POST	Authenticate a customer or administrator.	200 OK; 401 invalid credentials.
/api/books	GET	List/search available books.	200 OK; 500 server error.
/api/cart	POST / PUT / DELETE	Add or update cart items.	200/201; 400 invalid quantity; 409 stock issue.
/api/orders	POST	Create an order after checkout/payment flow.	201 Created; 400 invalid data; 409 payment/stock issue.
/api/orders/{id}	GET	View an authorised user's order.	200 OK; 403/404 if inaccessible.
/api/admin/orders/{id}/status	PUT	Update order status for an administrator.	200 OK; 403 unauthorized; 404 not found.
/api/reviews	POST	Submit a rating/review for an eligible purchase.	201 Created; 400 validation error; 403 not eligible.

4.4 Error Handling, Logging & Monitoring

- Use consistent HTTP status codes and short messages that users can understand.
- Validate input at the server even when client-side validation is present.
- Do not expose stack traces, passwords, JWTs, payment secrets or other sensitive information in responses or logs.
- Use clear error states for failed payment, unavailable stock, invalid login and unauthorised administrator access.
- Record useful operational events such as authentication failures, order failures and administrator actions without logging secrets.

- Monitor basic application health, API errors and order/payment failure rates during testing or deployment.

4.5 UX Design

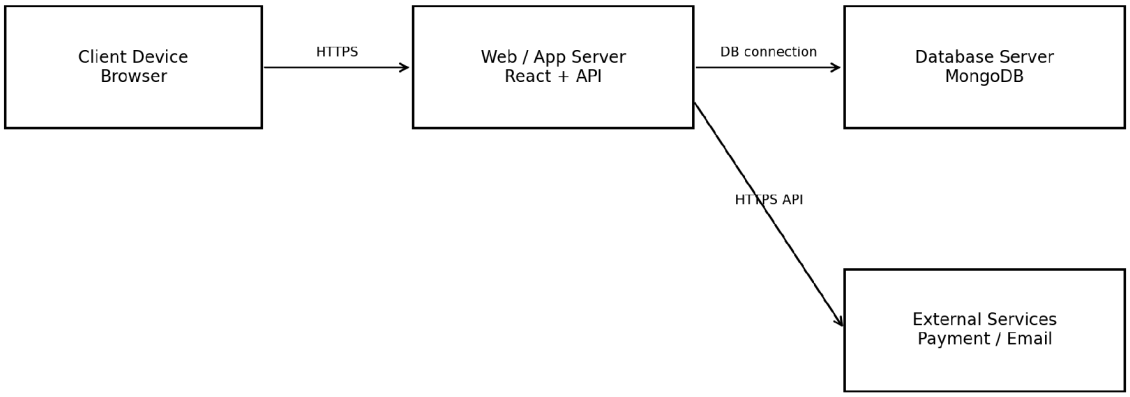
The customer interface should keep the main path simple: browse/search → view book → add to cart → checkout → payment → order confirmation. The administrator interface should use separate protected pages for catalogue, inventory, orders, users and reviews.

- Use clear labels and validation messages on forms.
- Keep important order information visible before payment confirmation.
- Show stock and availability clearly.
- Provide readable layouts on desktop and common mobile screen sizes.
- Do not expose technical error messages to normal users.

4.6 Deployment View

The deployment view below represents a simple project deployment. The client communicates with the application server over HTTPS. The application server communicates with the database and required external services.

Online Bookstore – Deployment View



4.7 Architecture Decision Records

ADR	Decision	Reason
ADR-01	Use layered architecture.	It keeps the project understandable and separates UI, API, business rules and data access.
ADR-02	Use REST/JSON APIs.	It is straightforward to implement and works well between a web frontend and backend.
ADR-03	Use MongoDB for application data.	The SRS already identifies MongoDB as the database choice and the bookstore entities fit a document-oriented model.
ADR-04	Keep payment as an external service.	Payment processing should not be implemented inside the bookstore application; a gateway/sandbox is easier to test and replace.
ADR-05	Use server-side RBAC.	Admin privileges must be enforced by the backend and cannot depend only on hiding UI options.

5. Appendices

5.1 Glossary

Term	Meaning
API	Application Programming Interface.
ADR	Architecture Decision Record.
JWT	JSON Web Token used for authenticated requests.
RBAC	Role-Based Access Control.
REST	Style used for resource-based web APIs.
SRS	Software Requirements Specification.
STP	Software Test Plan.
RTM	Requirements Traceability Matrix.
UML	Unified Modeling Language.

5.2 References

- Online Bookstore SRS v1.0.
- Online Bookstore Software Test Plan v1.0.
- SAD Template – Software Architecture and Design Specification.
- UML lecture/reference material supplied for the project, including use-case, component and sequence diagram conventions.

5.3 Tools

- VS Code – implementation and debugging.
- Git/GitHub – source control and collaboration.
- Postman – API testing.
- draw.io / PlantUML-style UML notation – architecture and interaction diagrams.
- MongoDB tools – database inspection and testing.